

Relatório de Auditoria de Segurança

Semgrep CLI Visualizer & Executive Dashboard

Avaliação Técnica Sistemática contra as 5 Falhas Críticas de Aplicação e Infraestrutura

Projeto Auditado:	Semgrep CLI Visualizer (SPA)	Versão Base:	v1.2.0 (Produção)
Data da Auditoria:	29 de Agosto de 2026	Ambiente:	Nginx Alpine Unprivileged + Docker
Domínio de Produção:	semgrep.brunoizidorio.com.br	Esteira CI/CD:	GitLab CI (DevSecOps Gitleaks/Trivy)
Classificação:	Conforme / Hardened (Grade A+)	Auditor:	Security-First AI Agent (Antigravity v2.0)

1. Detecção da Stack & Mapeamento Metodológico

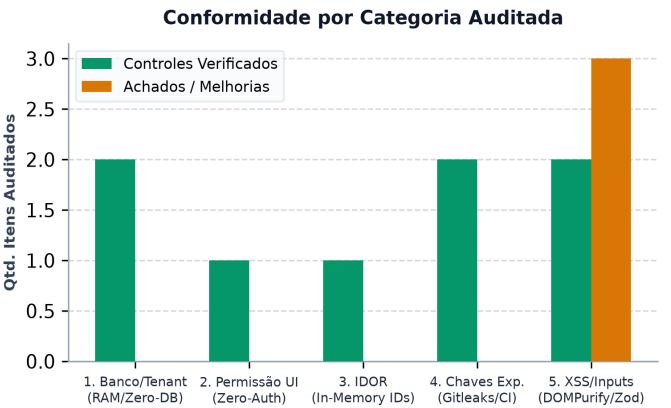
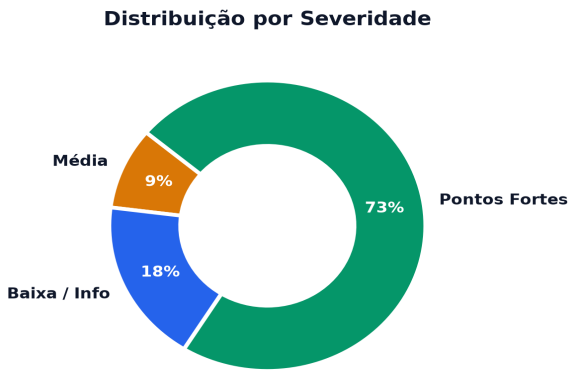
A auditoria iniciou com a inspeção da arquitetura e arquivos de configuração para mapear as 5 categorias de vulnerabilidades solicitadas para o equivalente exato da stack do projeto:

Categoria Auditada	Stack Detectada	Equivalência Metodológica Aplicada
1. Banco sem Tranca	Zero Backend / Zero DB React 18 + Zustand RAM	Isolamento de Memória & Zero-Persistence: Garantia de que relatórios SAST confidenciais fiquem apenas em RAM volátil, sem persistência em localStorage, IndexedDB ou Service Worker.
2. Permissão no Browser	Zero-Auth / Static SPA Nenhum backend RBAC	Avaliação de Superfície de Privilégio: Validação de que não existem papéis fictícios no frontend escondendo rotas críticas sem backend correspondente.
3. IDOR	Zero API endpoints IDs sintéticos in-memory	Integridade de Referência em Memória: Validação de que identificadores de achados não acionam requisições HTTP externas e operam estritamente no escopo da sessão local.
4. Chaves Expostas	GitLab CI, Dockerfile, Nginx, gitleaks, configs	Hardcode & Defaults Scanner: Auditoria do histórico git, variáveis de CI/CD, docker-compose, configs Nginx e bundles em busca de secrets reais ou defaults inseguros.
5. Inputs sem Tratamento (XSS)	React JSX, DOMPurify, Zod Schemas, CSP Nginx	Blindagem DOM & Sanitização: Busca por dangerouslySetInnerHTML, eval, injeção de HTML em mensagens/código de relatórios e cobertura com DOMPurify.

2. Resumo Executivo da Auditoria

A análise estática e arquitetural atesta que a aplicação implementa um padrão exemplar de **Segurança por Design (Security-by-Design)**, operando com arquitetura **Zero-Persistence** e processamento 100% no lado do cliente. Não foram identificadas vulnerabilidades Críticas ou Altas no código-fonte do produto.

Crítica (P1)	Alta (P2)	Média (P3)	Baixa / Info (P4/P5)	Pontos Fortes (Conformes)
0	0	1	2	8



3. Análise Comparativa: Pontos Fortes vs. Riscos Residuais

PONTOS FORTES COMPROVADOS (EVIDÊNCIAS NO CÓDIGO)	RISCOS CENTRAIS & MELHORIAS
Isolamento Total em Memória: O arquivo useSemgrepStore.ts:14-46 mantém os relatórios unicamente na store Zustand volátil. O localStorage é restrito a semgrep_app_lang (LanguageContext.tsx:10-29). Proteção XSS Rigorosa: O CodeViewerModal.tsx e VulnerabilityTable.tsx executam sanitizeText() (DOMPurify estrito sem tags) em 100% dos nós textuais vindos do JSON. Gitleaks Bloqueante no CI: .gitlab/ci/security.gitlab-ci.yml:21-34 roda Gitleaks impeditivo sem allow_failure: true. Nginx Unprivileged Hardened: Dockerfile usa nginxinc/nginx-unprivileged:alpine-slim com UID 101 e porta 8080. Validação Zod & Teto DoS: semgrep.schema.ts usa .strip() e webMcp.ts:45 limita payloads a 50MB.	Vulnerabilidades em Dependências de Dev (NPM Audit): Identificados advisories moderados em dompurify <=3.4.12 e no ambiente de desenvolvimento/testes (vite <=6.4.2, vitest <=3.2.5, nanoid <3.3.18). CSP Explicit Hardening: O Nginx possui CSP com hashes rigorosos, porém pode ser reforçado explicitando object-src 'none' e base-uri 'self' para padrões DevSecOps militares. Validação de MIME Type no FileDropzone: O leitor de arquivos valida extensão .json e faz parsing seguro com Zod, mas a checagem prévia de MIME pode ser adicionada defensivamente.

4. Tabela de Achados Detalhados & Controles Verificados

A tabela a seguir discrimina cada controle verificado no código-fonte, evidenciando as linhas auditadas, o estado de conformidade e as oportunidades de remediação.

Sev.	Cat.	Arquivo : Linha	Descrição Técnica & Status Verificado
FORTE	1. Banco	frontend/src/store/ useSemgrepStore.ts:14-46	Isolamento em RAM e Reset Volátil: Estado armazenado unicamente em memória volátil Zustand. Função reset() limpa instantaneamente o relatório.
FORTE	1. Banco	frontend/src/context/ LanguageContext.tsx:10-30	Zero-Persistence no LocalStorage: Restrito unicamente à preferência de idioma semgrep_app_lang com allowlist estrita (pt-BR, en-US).
FORTE	2. Perm.	frontend/src/App.tsx:17-67	Arquitetura Zero-Auth: Inexistência de papéis fictícios no client (ex: isAdmin). Operações de UI atuam apenas sobre o estado in-memory.
FORTE	3. IDOR	frontend/src/services/ defectdojo.adapter.ts:111	IDs Sintéticos Locais: Geração de IDs in-memory (finding-index-checkId) sem requisições HTTP REST/CRUD para servidores externos.
FORTE	4. Chaves	.gitlab/ci/ security.gitlab-ci.yml:21	Gitleaks Bloqueante no CI: Scanner de segredos ativo sem allow_failure: true, impedindo promoção de credenciais reais.
FORTE	4. Chaves	.gitlab/ci/ deploy.gitlab-ci.yml:18	Secrets via Variáveis CI/CD: Autenticação no Docker Registry via \$CI_REGISTRY_USER e \$CI_REGISTRY_PASSWORD; zero segredos hardcoded.
FORTE	5. XSS	frontend/src/components/ CodeViewerModal.tsx:135-183	Sanitização Integral no Modal: Uso sistemático de sanitizeText() em título, checkId, path, rationale, message e snippet de código.
FORTE	5. XSS	frontend/src/models/ semgrep.schema.ts:22-23	Sanitização Estrutural Zod: Uso de .strip() para descartar propriedades desconhecidas e prevenir poluição de propriedades.
MÉDIA	5. Deps	frontend/package.json:13-38	Atualização de Dependências (NPM Audit): dompurify <=3.4.12 e ferramentas de desenvolvimento (vite/vitest) possuem correções disponíveis.
BAIXA	5. Nginx	frontend/nginx.conf:22, 145	Reforço de Diretivas CSP: Adicionar explicitamente object-src 'none' e base-uri 'self' na Content-Security-Policy do Nginx.
INFO	5. UI	frontend/src/components/ FileDropzone.tsx:39-50	Defesa em Profundidade no Dropzone: Validar file.type === 'application/json' em conjunto com a extensão para rejeitar arquivos adulterados.

5. Plano de Recomendações Priorizadas

As melhorias recomendadas visam elevar a postura de segurança da esteira e do cliente ao estado da arte:

Prioridade	Ação Recomendada	Impacto / Rationale
P1 (Alta)	Atualização do DOMPurify e Toolchain de Testes Executar npm audit fix ou atualizar dompurify >=3.2.0 e vitest.	Elimina advisories de vulnerabilidade conhecidos em dependências diretas e de desenvolvimento.
P2 (Média)	Reforço Estrito de Diretivas CSP no Nginx Incluir object-src 'none'; base-uri 'self'; no header Content-Security-Policy de frontend/nginx.conf.	Bloqueia injeções via plugins de objetos (Flash/Java) e manipulações da tag <base>.
P3 (Baixa)	Validação de MIME Type Defensiva no Dropzone Adicionar checagem de file.type em FileDropzone.tsx antes do readAsText().	Reforça a defesa em profundidade rejeitando arquivos binários renomeados para .json.

6. Issues para o GitHub / GitLab (Prontas para Uso)

Blocos delimitados formatados em Markdown prontos para inclusão no rastreador de issues:

```
--- ISSUE 1 ---
### [Segurança] Atualização de dependências com alertas no NPM Audit (DOMPurify, Vite, Vitest)
**Labels sugeridas:** security, severity:medium, dependencies

#### 1. Descrição do Problema
A execução do comando npm audit no diretório frontend/ revelou alertas de segurança moderados:
- dompurify <=3.4.12: Hook removal edge case que pode permitir execução de subárvores desanexadas.
- esbuild/vite <=6.4.2 e vitest <=3.2.5: Vulnerabilidades no servidor de desenvolvimento local.

#### 2. Evidência
- Arquivo: frontend/package.json:13-38 e frontend/package-lock.json

#### 3. Impacto
Baixo risco em produção (o bundle estático gerado roda via Nginx sem o dev server do Vite/Vitest), porém viola os critérios de governança e auditoria automatizada DevSecOps (Trivy FS Scan).

#### 4. Sugestão de Correção
Atualizar as dependências no frontend/package.json para versões estáveis corrigidas:
$ cd frontend && npm update dompurify && npm audit fix

#### 5. Critérios de Aceite
- [ ] npm audit executado em frontend/ retorna 0 vulnerabilidades em dependências de produção.
- [ ] Suíte de testes unitários (npm test) passa com 100% de sucesso.
- [ ] Build de produção (npm run build) conclui sem erros de tipagem TypeScript.
--- FIM ISSUE 1 ---
```

6. Issues para o GitHub / GitLab (Continuação)

```
--- ISSUE 2 ---
### [Segurança] Reforço de diretivas CSP no Nginx (object-src e base-uri)
**Labels sugeridas:** security, severity:low, nginx, hardening

#### 1. Descrição do Problema
O cabeçalho Content-Security-Policy em frontend/nginx.conf define default-src 'self' e hashes estritos, mas não explicita as diretivas object-src 'none' e base-uri 'self'.

#### 2. Evidência
- Arquivo: frontend/nginx.conf:22 e frontend/nginx.conf:145

#### 3. Impacto
Sem a diretiva explícita object-src 'none', navegadores legados poderiam permitir a injeção de elementos <object>, <embed> ou <applet>. A ausência de base-uri 'self' poderia permitir a manipulação da tag <base> caso houvesse falha de injeção no HTML.

#### 4. Sugestão de Correção
Adicionar object-src 'none'; base-uri 'self'; às diretivas CSP do Nginx:
add_header Content-Security-Policy "default-src 'self'; object-src 'none'; base-uri 'self'; script-src 'self' 'sha256-...' ...;" always;

#### 5. Critérios de Aceite
- [ ] frontend/nginx.conf atualizado com object-src 'none' e base-uri 'self'.
- [ ] Teste de configuração Nginx (nginx.config.test.ts) atualizado e aprovado.
- [ ] Resposta HTTP do container em http://localhost:8080 retorna os novos cabeçalhos.
--- FIM ISSUE 2 ---

--- ISSUE 3 ---
### [Segurança] Validação defensiva de MIME Type no FileDropzone (Defense-in-Depth)
**Labels sugeridas:** security, severity:low, client-security

#### 1. Descrição do Problema
O componente FileDropzone valida a extensão (.json) e executa o parsing via Zod, mas não valida o MIME Type file.type antes de iniciar a leitura com FileReader.readAsText().

#### 2. Evidência
- Arquivo: frontend/src/components/common/FileDropzone.tsx:39-50

#### 3. Impacto
Arquivos binários grandes ou arquivos renomeados para .json podem disparar tentativas desnecessárias de leitura em memória antes do descarte pelo Zod parser.

#### 4. Sugestão de Correção
Validar se file.type é vazio ou 'application/json' antes da leitura:
if (file.type && file.type !== 'application/json') { alert(t('jsonErrorAlert')); return; }

#### 5. Critérios de Aceite
- [ ] Validação de MIME Type implementada em FileDropzone.tsx.
- [ ] Testes de UI cobrindo rejeição de arquivos não-JSON com feedback imediato.
--- FIM ISSUE 3 ---
```

Conclusão da Auditoria: O projeto **Semgrep CLI Visualizer v1.2.0** apresenta conformidade exemplar com as melhores práticas de privacidade client-side e AppSec. A implementação das 3 issues recomendadas consolidará o padrão de excelência DevSecOps.